

Project: Sliding Tile Puzzle

(draft)

Alexandre Miguel Fernandes Dias

September 7, 2026

Contents

Preface	2
1 Application development	3
2 Solution algorithm	5

Preface

When I was a child, probably in 1998, I got for Christmas the Game Boy game “[The Hunchback of Notre Dame: Topsy Turvy Games](#)”. This game is a series of mini-games, with one of them being a sliding tile puzzle game. I played it enough times, to the point of devising an algorithm to solve the puzzles. At some point, already after I had learned Python and graphical user interface (GUI) programming (2018–2023), I wanted to develop an application myself to solve this kind of puzzle and to implement my algorithm so I could watch these puzzles being solved programmatically. The application developed in this project is the result of such a will.

Further documentation:

- Demo video [[YouTube](#)]
- Code repository [[GitHub](#)]

Chapter 1

Application development

The developed application has the following features and functionalities:

- Generates puzzles having n^2 tiles, where n is both the number of rows and the number of columns (“square” puzzles):
 - Only solvable puzzles are presented to the user to the solved by default (unsolvable puzzles are skipped); this behavior can be easily changed through a function argument, so that unsolvable puzzles are shown and not skipped
 - The tile of the lower right corner, that is, the tile in the last row and last column (row n , column n) is considered by default as the missing tile
 - An image is laid on top of the puzzle tiles, except for the missing tile; to ensure compatibility between puzzle size (n rows and n columns) and image size (pixel height and pixel width) a border is added if required to ensure integer divisibility, that is, that image height and width plus eventually added border height and width and both divisible by the number of rows and columns (n)
- Allows solving the currently shown puzzle:
 - The keyboard arrow keys (Left/Right, Up/Down) can be pressed to move the missing tile, by default, in all directions within the boundaries of the puzzle; an option is given to the user to solve the puzzle in a wrap-around fashion, that is, if the missing tile is in the last/first row (column) and is moved down/up (right/left), it will end up in the first/last row (column), as if the first and last rows and first and last columns were adjacent to each other
 - A button allows the current puzzle to be automatically solved when clicked
 - Clicking another button leads to the current puzzle being automatically solved and for the application to generate and automatically solve new puzzles *ad infinitum*; when generating and solving puzzles *ad infinitum*, two plots are shown, one with the ... as a function of the number of moves and ...
 - Clicking one of the previous buttons after puzzle solving has been initiated allows interrupting the solving of the current puzzle
- The moves of the missing tile so far (since the generation of the current puzzle) can be undone, that is, the puzzle solving progress can be reset
- Along with the puzzle being solved, the application also displays a solved version of the puzzle and a heatmap representing how far each tile is from its final position, with

red (RGB=(255,0,0)) meaning the maximum possible distance ($n\sqrt{2}$ tiles, the diagonal size of the puzzle), yellow (RGB=(255,255,0)) half of that distance ($n\sqrt{2}/2$), and green (RGB=(0,255,0)) for zero distance; for tiles not yet in their final positions, all intermediate colors between red and yellow and between yellow and green can be shown, and black is shown for tiles in their final positions; both the solved version of the puzzle and the heatmap are intended to visually help the user solving the puzzle

- The puzzle number (i -th solvable puzzle generated so far) as well as the number of moves so far of the puzzle solving progress are displayed

In Figure 1.1, the developed application is shown/illustrated/depicted.

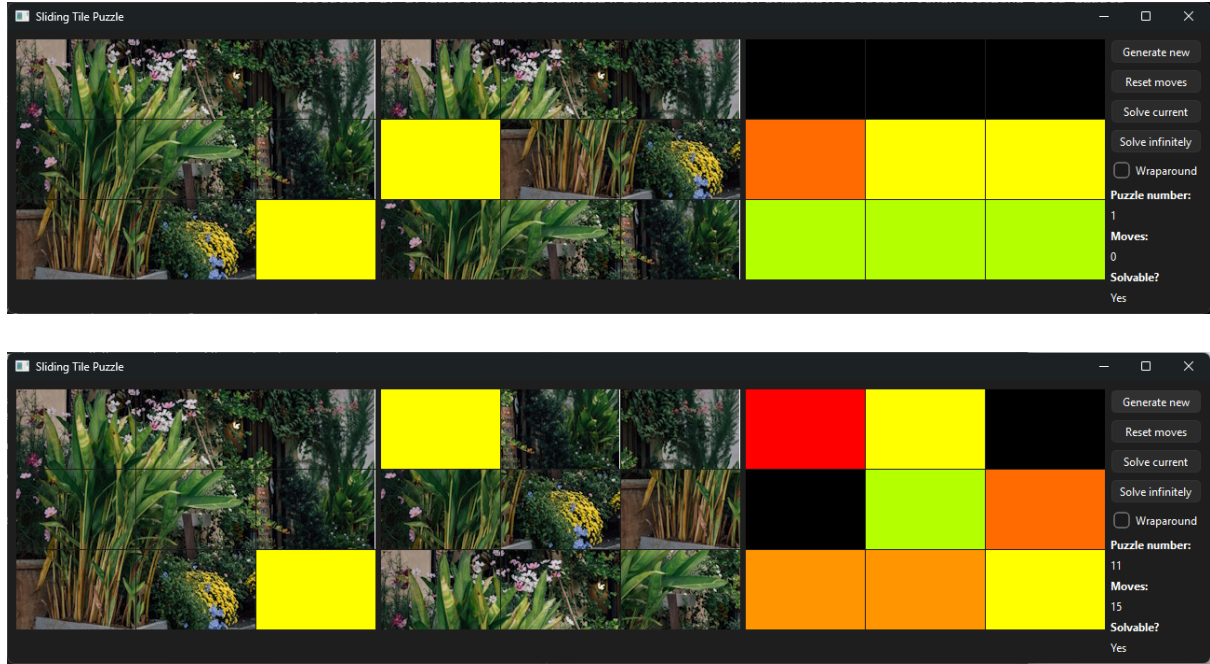


Figure 1.1: Screenshots of the developed application.

Chapter 2

Solution algorithm

The implemented algorithm solves a puzzle from top to bottom, one row at a time, and each row from left to right, one tile at a time, except for two special cases: when solving the last two tiles of the row (last two columns) and when solving the last two rows. This is depicted in Figure 2.1.

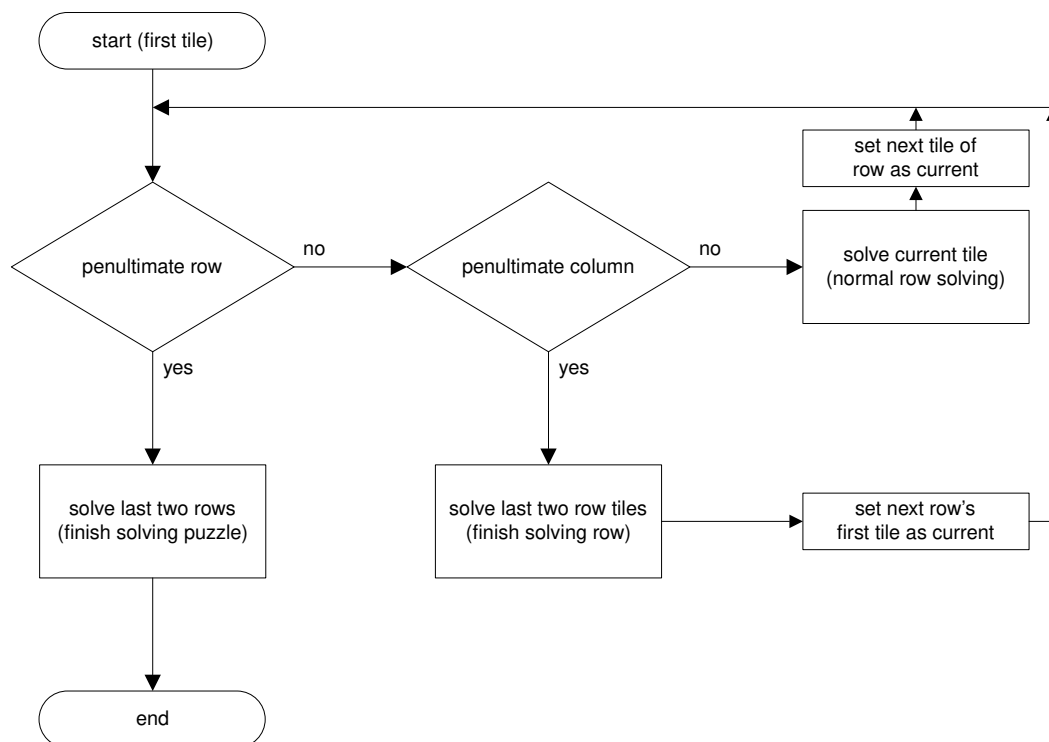


Figure 2.1: Flowchart of the implemented algorithm with its main solving parts.

(to be continued...)